

LAPORAN PRAKTIKUM PEKAN 3
PEMOGRAMAN BERORIENTASI OBJEK

Keamanan & Integritas Data (Enkapsulasi)



Oleh:
Naysha Aprilia Rizki (2511531014)
Kelas B

Dosen Pengampu:
Nurfiah, S.ST., M.Kom.

Asisten Praktikum:
Ezza Addini

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
PADANG
2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam Pemrograman Berorientasi Objek (PBO), keamanan dan integritas data merupakan hal yang penting dalam pengembangan sebuah program. Data yang terdapat di dalam suatu class sebaiknya tidak dapat diakses atau dimanipulasi secara langsung dari luar class karena dapat menyebabkan perubahan data yang tidak sesuai dengan aturan program. Oleh karena itu, diperlukan suatu mekanisme untuk membatasi akses terhadap data tersebut.

Salah satu konsep yang digunakan adalah enkapsulasi dan Information Hiding. Enkapsulasi memungkinkan atribut dalam suatu class dilindungi menggunakan access modifier seperti `private`, `protected`, dan `public`. Selain itu, getter dan setter dapat digunakan untuk mengatur proses pembacaan dan perubahan data.

Pada praktikum ini, konsep keamanan data diterapkan pada program Mini Bank dengan mengamankan atribut pada class Transaksi dan Rekening. Selain itu, ditambahkan PIN sebagai sistem autentikasi yang digunakan sebelum melakukan tarik tunai dan melihat riwayat transaksi.

1.2 Tujuan

1. Memahami konsep Information Hiding dalam Pemrograman Berorientasi Objek.
2. Memahami penggunaan access modifier `private`, `protected`, dan `public`.
3. Menerapkan konsep enkapsulasi menggunakan getter dan setter.
4. Mengamankan data pada class Transaksi dan Rekening.
5. Menerapkan autentikasi PIN pada program Mini Bank.

1.3 Manfaat

1. Memahami pentingnya keamanan dan integritas data dalam program.
2. Mampu menerapkan access modifier untuk membatasi akses terhadap atribut.
3. Mampu menerapkan enkapsulasi pada program Java.
4. Memahami penggunaan getter, setter, dan autentikasi PIN.
5. Meningkatkan kemampuan dalam membuat program yang memiliki keamanan data lebih baik.

BAB II

PEMBAHASAN

2.1 Teori

Dalam Pemrograman Berorientasi Objek (PBO), keamanan data diperlukan agar atribut dalam suatu class tidak dapat dimanipulasi secara langsung dari luar class. Konsep yang digunakan untuk membatasi akses tersebut adalah **Information Hiding** dan **enkapsulasi**.

- **Information Hiding**, merupakan konsep untuk menyembunyikan detail implementasi internal suatu object sehingga data tidak dapat diakses secara langsung. Object lebih mengutamakan penggunaan method sebagai behavior daripada memberikan akses langsung terhadap variabel.

- **Access Modifier**, digunakan untuk mengatur tingkat akses atribut dan method. `private` hanya dapat diakses dalam class, `protected` dapat diakses dalam package dan subclass, sedangkan `public` dapat diakses dari mana saja.

- **Getter dan Setter**, digunakan sebagai perantara untuk mengakses atribut yang bersifat `private`. Getter digunakan untuk membaca data, sedangkan setter digunakan untuk mengubah data.

- **Enkapsulasi**, merupakan konsep untuk membatasi akses langsung terhadap data dalam class dengan melindungi atribut menggunakan access modifier dan menyediakan method yang diperlukan untuk mengakses data tersebut.

- **Autentikasi PIN**, digunakan untuk memeriksa PIN yang dimasukkan pengguna sebelum menjalankan fungsi tertentu. Pada praktikum ini, autentikasi digunakan pada proses tarik tunai dan cetak mutasi. Jika PIN benar, proses dilanjutkan, sedangkan jika salah maka akses ditolak.

2.2 Langkah Langkah

1. Membuat Class Transaksi

- 1) Pertama saya membuat Package “prak3_enkapsulasi”
- 2) Selanjutnya dilanjutkan dengan membuat Class Transaksi, Berikut Screenshot lengkap nya

```
1 package prak3_enkapsulasi;
2
3 public class Transaksi {
4     //mengubah atribut id private
5     private String idTransaksi;
6     private String jenis;
7     private double nominal;
8
9     public Transaksi(String id, String jenis, double nominal) {
10         this.idTransaksi = id;
11         this.jenis = jenis;
12         this.nominal = nominal;
13     }
14     // Hanya menyediakan getter (Read Only)
15     public String getIdTransaksi() { return idTransaksi; }
16     public String getJenis() { return jenis; }
17     public double getNominal() { return nominal; }
18
19     public void cetakDetail() {
20         System.out.println("ID: " + idTransaksi + " | Jenis:"
21             + jenis + " | Nominal: Rp" + nominal);
22     }
23 }
24 }
```

Penjelasan:

- 1) Buat class Transaksi

```
public class Transaksi {
```

Ini mengartikan kita sedang membuat sebuah class bernama Transaksi. Class bisa dianggap sebagai cetakan/model untuk membuat objek Transaksi.

- 2) Membuat State/Atribut

```
//mengubah atribut id private
private String idTransaksi;
private String jenis;
private double nominal;
```

Atribut ini merupakan variable yang menyimpan informasi tentang suatu objek. Pada class transaksi ini tersimpan data berupa

- idTransaksi dengan tipe data string

- jenis dengan tipe data string
- nominal dengan tipe data double.

Kata private digunakan agar atribut hanya dapat diakses secara langsung dari dalam class Transaksi. Hal ini merupakan penerapan konsep **enkapsulasi**.

3) Buat Constructor

```
public Transaksi(String id, String jenis, double nominal) {
    this.idTransaksi = id;
    this.jenis = jenis;
    this.nominal = nominal;
}
```

Constructor ini digunakan untuk menginisialisasikan atau memberi nilai awal pada objek Transaksi Ketika objek dibuat.

- idTransaksi di inisialisasikan dengan nilai id
- jenis diinisialisasikan dengan nilai jenis transaksi, yaitu jenis transaksi debit atau kredit
- nominal diinisialisasikan dengan nilai nominal transaksi

this digunakan untuk menunjukkan atribut yang berada di dalam class Transaksi.

4) Membuat Method Getter

```
// Hanya menyediakan getter (Read Only)
public String getIdTransaksi() { return idTransaksi;}
public String getJenis() { return jenis; }
public double getNominal() { return nominal;}
```

Method getter digunakan untuk mengambil atau membaca nilai atribut yang bersifat private. Pada class Transaksi terdapat tiga getter, yaitu:

- getIdTransaksi() digunakan untuk mengambil nilai idTransaksi.
- getJenis() digunakan untuk mengambil nilai jenis.
- getNominal() digunakan untuk mengambil nilai nominal.

Getter digunakan karena atribut dibuat private, sehingga nilai atribut tidak dapat diakses secara langsung dari luar class

5) Buat Method cetakDetail

```
public void cetakDetail() {  
    System.out.println("ID: " + idTransaksi + " | Jenis:"  
        + jenis + " | Nominal: Rp" + nominal);  
}  
}
```

Method **cetakDetail()** digunakan untuk menampilkan detail transaksi yang berisi ID transaksi, jenis transaksi, dan nominal transaksi. Program akan menampilkan informasi dengan format “ID: (ID transaksi) | Jenis: (jenis transaksi) | Nominal: Rp (nominal)”.

2. Membuat Class Rekening

- 3) Salin Code Program yang sudah dibuat pada praktikum prak2_Relasi, Dan ubah atau tambah beberapa bagian saja. Berikut Screenshot lengkap nya

```
1 package prak3_enkapsulasi;  
2 import java.util.ArrayList;  
3  
4 public class Rekening {  
5     // Mengunci atribut dengan private  
6     private String nomorRekening;  
7     private String namaPemilik;  
8     private double saldo;  
9     private String pin; //Data Sensitif  
10  
11     private ArrayList<Transaksi> riwayatTransaksi;  
12  
13     //2.Modifikasi Constructor untuk menerima PIN awal  
14     public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {  
15         this.nomorRekening = nomor;  
16         this.namaPemilik = nama;  
17         this.saldo = saldoAwal;  
18  
19         //validasi PIN dalam Constructor  
20         if (pinAwal.length() == 6) {  
21             this.pin = pinAwal;  
22         } else {  
23             System.out.println("Peringatan: PIN harus 6 digit Menggunakan PIN default 123456");  
24             this.pin = "123456";  
25         }  
26         this.riwayatTransaksi = new ArrayList<>();  
27         System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");  
28     }  
29     //3. Getter untuk atribut yang diizinkan dibaca publik  
30     public String getNomorRekening() { return nomorRekening; }  
31     public String getNamaPemilik() { return namaPemilik; }  
32  
33     //4. Method Otentikasi Internal (Validasi Enkapsulasi)  
34     public boolean otentikasi(String inputPin) {  
35         return this.pin.equals(inputPin);  
36     }  
37 }
```

Pada bagian ini yang di ubah dan ada tambahan, berikut penjelasannya:

1) Mengunci Atribut dengan Private

```
public class Rekening {  
    // Mengunci atribut dengan private  
    private String nomorRekening;  
    private String namaPemilik;  
    private double saldo;  
    private String pin; //Data Sensitif  
  
    private ArrayList<Transaksi> riwayatTransaksi;  
}
```

Atribut merupakan variabel yang digunakan untuk menyimpan informasi dari suatu objek. Pada class Rekening terdapat beberapa atribut, yaitu:

- nomorRekening dengan tipe data **String**, digunakan untuk menyimpan nomor rekening.
- namaPemilik dengan tipe data **String**, digunakan untuk menyimpan nama pemilik rekening
- saldo dengan tipe data **double**, digunakan untuk menyimpan jumlah saldo rekening.
- pin dengan tipe data **String**, digunakan untuk menyimpan PIN rekening. PIN termasuk data sensitif.
- riwayatTransaksi dengan tipe **ArrayList<Transaksi>**, digunakan untuk menyimpan daftar transaksi yang dilakukan pada rekening.

Semua atribut menggunakan private agar data tidak dapat diakses atau diubah secara langsung dari luar class. Hal ini merupakan penerapan konsep **enkapsulasi**.

2) Modifikasi Constructor untuk menerima PIN awal

```
//2.Modifikasi Constructor untuk menerima PIN awal
public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
    this.nomorRekening = nomor;
    this.namaPemilik = nama;
    this.saldo = saldoAwal;
}
```

Constructor digunakan untuk memberikan nilai awal pada objek Rekening ketika objek tersebut dibuat.

- nomorRekening diinisialisasikan dengan nilai dari parameter nomor
- nomorRekening diinisialisasikan dengan nilai dari parameter nomor
- saldo diinisialisasikan dengan nilai dari parameter saldoAwal.

Validasi PIN dalam Constructor

```
//validasi PIN dalam Constructor
if (pinAwal.length() != 6) {
    this.pin = pinAwal;
} else {
    System.out.println("Peringatan: PIN harus 6 digit Menggunakan PIN default 123456");
    this.pin = "123456";
}
this.riwayatTransaksi = new ArrayList<>();
System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
}
```

Bagian ini digunakan untuk melakukan **validasi PIN** ketika rekening dibuat.

- Jika panjang PIN sama dengan 6 karakter, maka PIN tersebut akan digunakan.
- Jika PIN tidak berjumlah 6 karakter, program akan memberikan peringatan.
- Kemudian program menggunakan PIN default yaitu 123456

Dengan adanya validasi ini, data PIN yang disimpan dapat dikontrol sejak awal pembuatan objek

this.riwayatTransaksi = new ArrayList<>(); Kode ini digunakan untuk membuat sebuah ArrayList baru yang akan digunakan untuk menyimpan objek-objek Transaksi.

3) Getter untuk atribut yang diizinkan dibaca public

```
//3. Getter untuk atribut yang diizinkan dibaca public
public String getNomorRekening() { return nomorRekening; }
public String getNamaPemilik() { return namaPemilik; }
```

Method getter digunakan untuk **mengambil atau membaca nilai atribut** yang bersifat private.

Pada class Rekening terdapat dua getter:

- getNomorRekening() digunakan untuk mengambil nomor rekening.
- getNamaPemilik() digunakan untuk mengambil nama pemilik rekening.

Tidak dibuat getter untuk pin karena PIN merupakan **data sensitif** dan tidak boleh sembarangan diakses dari luar class.

4) Method Otentikasi internal (Validasi Enkapsulasi)

```
//4. Method Otentikasi Internal (Validasi Enkapsulasi)
public boolean otentikasi(String inputPin) {
    return this.pin.equals(inputPin);
}
```

Method otentikasi() digunakan untuk memeriksa apakah PIN yang dimasukkan sesuai dengan PIN rekening.

- inputPin merupakan PIN yang dimasukkan oleh pengguna.

- `this.pin` merupakan PIN yang tersimpan di dalam rekening.
- `equals()` digunakan untuk membandingkan kedua PIN.
- Method mengembalikan nilai `true` jika PIN sesuai dan `false` jika PIN tidak sesuai.

Dengan cara ini, PIN tetap tersimpan secara private dan tidak perlu diberikan langsung kepada class lain.

Pertanyaan

Mengapa tidak dibuat method `setSaldo()` dan `getPin()`?

Jawaban

Method `setSaldo()` tidak dibuat karena perubahan saldo harus dilakukan melalui method `setorTunai()` dan `tarikTunai()` agar tetap melalui proses dan validasi yang telah ditentukan. Sementara itu, method `getPin()` tidak dibuat karena PIN merupakan data sensitif yang harus dirahasiakan. Sebagai gantinya, digunakan method `otentikasi()` untuk memeriksa kesesuaian PIN tanpa menampilkan atau memberikan nilai PIN secara langsung.

TUGAS/LATIHAN

Perbarui program `Main.java` dengan aturan

- Menu 1 Sebelum menginstansiasi objek baru, minta user menginputkan PIN (String angka 6 digit).
- Keamanan Transaksi: Perbarui Menu 3 (Tarik Tunai) dan Menu 6 (Cetak Mutasi). Sebelum sistem mengeksekusi kedua method tersebut, sistem harus meminta user memasukkan PIN.
 - Panggil fungsi `akunAktif.otentikasi(pinYangDiinput)`.
 - Jika mengembalikan nilai `true`, jalankan transaksi.
 - Jika `false`, batalkan dan cetak: "Akses Ditolak: PIN yang Anda masukkan salah!".

Jawaban:

- Class `Main.java` Menu 1 (Buka Rekening)

```

switch(pilihan) {
    case 1:
        System.out.print("Masukan No Rekening: ");
        String no = input.nextLine();
        System.out.print("Masukan Nama Pemilik: ");
        String nama = input.nextLine();
        System.out.print("Masukan Saldo Awal: ");
        double saldo = input.nextDouble();
        input.nextLine();

        // minta input PIN
        System.out.print("Masukan PIN(6 digit):");
        String pin = input.nextLine();

        //Instansiasi Object /Menjalankan Constructor
        Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);

        daftarRekening.add(rekeningBaru);
        akunAktif = rekeningBaru;
        break;
}

```

Penjelasan

- **System.out.print("Masukan No Rekening: ");**
String no = input.nextLine();

Kode ini digunakan untuk meminta pengguna memasukkan nomor rekening. Nilai yang dimasukkan oleh pengguna disimpan ke dalam variabel no dengan tipe data String.

- **System.out.print("Masukan Nama Pemilik: ");**
String nama = input.nextLine();

Kode ini digunakan untuk meminta pengguna memasukkan nama pemilik rekening. Nama yang dimasukkan akan disimpan ke dalam variabel nama dengan tipe data String.

- **System.out.print("Masukan Saldo Awal: ");**
double saldo = input.nextDouble();
input.nextLine();

Kode ini digunakan untuk meminta pengguna memasukkan saldo awal rekening.

- saldo menggunakan tipe data double karena menyimpan nilai berupa angka.
- **input.nextDouble()** digunakan untuk membaca input berupa angka
- **input.nextLine()** digunakan untuk membersihkan sisa enter setelah penggunaan **nextDouble()** agar input berikutnya dapat terbaca dengan benar.

- **System.out.print("Masukan PIN(6 digit):");**

```
String pin = input.nextLine();
```

Kode ini digunakan untuk meminta pengguna memasukkan PIN rekening. PIN disimpan dalam variabel pin dengan tipe data String. PIN menggunakan String karena PIN merupakan kode, bukan nilai yang digunakan untuk perhitungan.

- **Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);**
daftarRekening.add(rekeningBaru);
akunAktif = rekeningBaru;

Kode tersebut digunakan untuk membuat objek rekening baru, menyimpannya ke dalam daftar rekening, dan menjadikannya sebagai akun aktif.

- **new Rekening(no, nama, saldo, pin)** digunakan untuk membuat objek Rekening baru dengan data nomor rekening, nama pemilik, saldo awal, dan PIN.
- **daftarRekening.add(rekeningBaru)** digunakan untuk menambahkan rekening baru ke dalam daftar rekening.
- **akunAktif = rekeningBaru** digunakan untuk menjadikan rekening yang baru dibuat sebagai akun yang sedang aktif digunakan.

OUTPUT

```
main (/Users/.../Application) on Program Files (x86) ...  
=== SISTEM PERBANKAN MINI ===  
  
Menu Utama:  
1.Buka Rekening Baru  
2.Setor Tunai  
3.Tarik tunai  
4.Cek Informasi Rekening  
5.Ganti Akun  
6.Cetak Mutasi (Riwayat)  
0.Keluar  
Pilih Menu:  
1  
Masukan No Rekening: 001  
Masukan Nama Pemilik: Naysha  
Masukan Saldo Awal: 200000  
Masukan PIN(6 digit):123456  
Rekening atas nama Naysha berhasil dibuat.
```

b. Class Main.java Menu 3 (Tarik Tunai) dan 6 (Cetak Mutasi)

a) Menu 3(Tarik Tunai)

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: Mohon Maaf, Anda belum memiliki nomor rekening!");
    } else {
        //minta pin
        System.out.print("Masukan PIN: ");
        String pinTarik = input.nextLine();
        //cek pin
        if (akunAktif.otentikasi(pinTarik)) {
            System.out.print("Masukan nominal tarik: ");
            double tarik = input.nextDouble();
            input.nextLine();
            akunAktif.tarikTunai(tarik);
        } else {
            System.out.print("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }
    break;
```

Penjelasan

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: Mohon Maaf, Anda belum memiliki nomor rekening!");
    } else {
```

Kode ini digunakan untuk mengecek apakah pengguna sudah memiliki akun aktif. Jika akunAktif null maka output akan keluar Error: Mohon Maaf, Anda belum memiliki nomor rekening!, namun jika akunAktif tidak null, maka program akan masuk ke bagian else dan melanjutkan proses penarikan.

```
//minta pin
System.out.print("Masukan PIN: ");
String pinTarik = input.nextLine();
```

Program meminta pengguna memasukkan PIN terlebih dahulu. PIN yang dimasukkan disimpan dalam variabel pinTarik, kemudian diperiksa menggunakan method:

- **akunAktif.otentikasi(pinTarik)**

Method otentikasi() akan membandingkan PIN yang dimasukkan dengan PIN yang tersimpan pada rekening. Jika PIN benar proses dilanjutkan namun jika salah akses ditolak.

```
//cek pin
if (akunAktif.otentikasi(pinTarik)) {
    System.out.print("Masukan nominal tarik: ");
    double tarik = input.nextDouble();
    input.nextLine();
    akunAktif.tarikTunai(tarik);
} else {
    System.out.print("Akses Ditolak: PIN yang Anda masukkan salah!");
}
}
break;
```

Kode ini digunakan untuk melanjutkan proses penarikan jika PIN benar dan menolak transaksi jika PIN salah.

- Jika **otentikasi(pinTarik)** menghasilkan true, program meminta pengguna memasukkan nominal penarikan.
- Nominal disimpan dalam variabel tarik dengan tipe data double.
- **akunAktif.tarikTunai(tarik)** digunakan untuk menjalankan proses penarikan pada rekening aktif.
- Jika PIN tidak sesuai, program masuk ke else dan menampilkan pesan "Akses Ditolak: PIN yang Anda masukkan salah!".

OUTPUT

```
Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
0.Keluar
Pilih Menu:
3
Masukan PIN: 123456
Masukan nominal tarik: 120000
Transaksi Berhasil
```

b) Menu 6 (Cek Mutasi)

```
case 6:
    if(akunAktif == null) {
        System.out.println("Error : Anda belum membuka rekening!");
    }else {
        //Masukan PIN
        System.out.print("Masukan PIN: ");
        String pinMutasi = input.nextLine();
        //Cek PIN
        if (akunAktif.otentikasi(pinMutasi)) {
            akunAktif.cetakMutasi();
        }else {
            System.out.print("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }
    break;
```

Penjelasan

```
if(akunAktif == null) {
    System.out.println("Error : Anda belum membuka rekening!");
}else {
```

Kode ini digunakan untuk mengecek apakah pengguna sudah memiliki rekening aktif. Jika `akunAktif == null`, berarti pengguna belum membuka rekening sehingga program menampilkan pesan error.

```
//Masukan PIN
System.out.print("Masukan PIN: ");
String pinMutasi = input.nextLine();
//Cek PIN
```

Program meminta pengguna memasukkan PIN dan menyimpannya ke variabel `pinMutasi`.

```
//Cek PIN
if (akunAktif.otentikasi(pinMutasi)) {
    akunAktif.cetakMutasi();
} else {
    System.out.print("Akses Ditolak: PIN yang Anda masukkan salah!");
}
}
break;
```

Kode ini digunakan untuk menampilkan riwayat transaksi jika PIN benar dan menolak akses jika PIN salah.

- Jika PIN benar, `akunAktif.cetakMutasi()` dipanggil untuk menampilkan seluruh riwayat transaksi pada rekening.
- Jika PIN salah, program menampilkan pesan "Akses Ditolak: PIN yang Anda masukkan salah!".

OUTPUT

```
Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
9.Keluar
Pilih Menu:
6
Masukan PIN: 123456
ID: TRX-T-1790521390192 | Jenis:Debit | Nominal: Rp120000.0
```

